AWS Fundamentals & Setup and Web Application Deployment

Lab Report with Output Screenshots

Name: _____ **Roll No:** _____

Course: _____ **Date:** _____

Objective: Understand cloud basics and the AWS environment, then deploy a real web application (WordPress) on EC2 backed by an Amazon RDS MySQL database.

Part 1: Fundamentals & Setup

1.1 Basic Cloud Concepts

Model	What you manage	AWS example	Everyday example
IaaS (Infrastructure as a Service)	Provider gives virtual servers, storage and networking. You manage the OS, software and data.	Amazon EC2, S3	Renting a virtual machine
PaaS (Platform as a Service)	Provider manages servers and runtime. You only deploy your code.	AWS Elastic Beanstalk	Heroku, Google App Engine
SaaS (Software as a Service)	Provider delivers a complete application over the internet. You just use it.	Amazon WorkMail, Chime	Gmail, Salesforce

1.2 Create an AWS Account

1. Open aws.amazon.com and choose **Create an AWS Account**.
2. Enter email, password and account name, then add contact and payment details (free tier applies).
3. Verify your phone number and choose the Basic (free) support plan.
4. Sign in to the console and enable MFA on the root user.

Screenshot 1

AWS Management Console home page after sign-in

Paste your screenshot here

1.3 Launch a Basic EC2 Instance

1. Console: **EC2 > Launch instance**. Name it, for example, *web-server-1*.
2. AMI: **Amazon Linux 2023**. Instance type: a free-tier eligible type (e.g. t3.micro).
3. Key pair: create a new key pair (.pem) and download it.
4. Network settings: allow **SSH (22)** from My IP and **HTTP (80)** from anywhere.
5. Click **Launch instance** and wait until the state is *Running*.

Screenshot 2

EC2 Instances list showing the instance in Running state

Paste your screenshot here

1.4 Connect to EC2 using SSH

```
chmod 400 mykey.pem
ssh -i mykey.pem ec2-user@<public-ip>
```

Screenshot 3

Terminal showing successful SSH login to the instance

Paste your screenshot here

1.5 Install a Web Server (Apache)

```
sudo dnf update -y
sudo dnf install -y httpd
sudo systemctl enable --now httpd
sudo systemctl status httpd
```

Screenshot 4

Output of systemctl status httpd showing active (running)

Paste your screenshot here

1.6 Host a Simple Webpage

```
echo "<h1>Hello from my EC2 instance</h1>" | sudo tee /var/www/html/index.html
```

Open `http://<public-ip>` in a browser.

Screenshot 5

Browser displaying the hosted webpage (address bar visible)

Paste your screenshot here

Outcome: AWS basics understood; an EC2 instance was launched, accessed over SSH, and used to host a webpage with Apache.

Part 2: Web Application Deployment

Architecture: Browser -> Elastic IP -> EC2 (Apache + PHP + WordPress) -> Amazon RDS (MySQL). The database is private and reachable only from the web server's security group.

2.1 Configure Security Groups

- **web-sg** (for EC2): SSH 22 from My IP; HTTP 80 and HTTPS 443 from anywhere.
- **rds-sg** (for RDS): MySQL/Aurora 3306 with source set to **web-sg** only.

Screenshot 6
web-sg inbound rules

Paste your screenshot here

Screenshot 7
rds-sg inbound rules (source = web-sg)

Paste your screenshot here

2.2 Configure Amazon RDS (MySQL)

1. Console: **RDS > Create database** > Standard create > **MySQL**.
2. Template: **Free tier**. Set master username (admin) and a password.
3. Connectivity: public access **No**, security group **rds-sg**.
4. Additional configuration: initial database name **wordpress**.
5. When the status is *Available*, copy the **endpoint**.

Screenshot 8

RDS database with status Available and its endpoint

Paste your screenshot here

2.3 Launch EC2 and Install WordPress

Launch a new EC2 instance with **web-sg**, connect over SSH, then run:

```
sudo dnf install -y httpd php php-mysqlnd php-fpm php-gd \
    php-mbstring php-xml mariadb105
sudo systemctl enable --now httpd
cd /tmp && wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz
sudo cp -r wordpress/* /var/www/html/
sudo chown -R apache:apache /var/www/html
```

Test that the server can reach the database:

```
mysql -h <rds-endpoint> -u admin -p
SHOW DATABASES;
```

Screenshot 9

MySQL login to RDS and SHOW DATABASES output (hide the password)

Paste your screenshot here

2.4 Assign an Elastic IP

Do this **before** running the WordPress installer, because WordPress stores the site URL during setup.

1. Console: **EC2 > Elastic IPs > Allocate Elastic IP address**.
2. Select it, choose **Actions > Associate Elastic IP address**, and pick the instance.

Screenshot 10

Elastic IP associated with the WordPress instance

Paste your screenshot here

2.5 Connect WordPress with RDS

```
cd /var/www/html
sudo cp wp-config-sample.php wp-config.php
sudo nano wp-config.php
```

Set these values in `wp-config.php`:

```
define('DB_NAME',     'wordpress');
define('DB_USER',     'admin');
define('DB_PASSWORD', '<your-password>');
define('DB_HOST',     '<rds-endpoint>');
```

Screenshot 11

wp-config.php showing DB_NAME and DB_HOST (blur the password)

Paste your screenshot here

2.6 Complete the WordPress Installation

Open `http://<elastic-ip>`, fill in the site title and admin account, and finish the installer.

Screenshot 12
WordPress installation / welcome screen

Paste your screenshot here

Screenshot 13
WordPress site homepage opened via the Elastic IP

Paste your screenshot here

Screenshot 14
WordPress admin dashboard (wp-admin)

Paste your screenshot here

Outcome: A WordPress application was hosted on EC2, connected to a private Amazon RDS MySQL database, secured with security groups and reachable through a static Elastic IP.

Clean-up (to avoid charges)

- Terminate both EC2 instances.
- Delete the RDS database.
- Release the Elastic IP (idle public IPv4 addresses are billed).